Documentation of DBDBS

Database interface for breeding programs

Table of contents

Important Note

There are a few commands that a user needs to run using the command line. These invoke the python 3 interpreter. The python 3 interpreter is called with different commands on different systems. On windows, it is often called with `py`, whereas on linux it is often called with `python3`. On many systems, it is often called with `python`. In this document, examples will call the python 3 interpreter with `python`. So, depending on your system, you may need to replace this word in the command. For example, instead of the command written in the documentation of `python -m dbdbs`, you may need to run the command `py -m dbdbs`.

Overview

Purpose

The purpose of DBDBS is to store, organize, and make accessible data to inform breeding decisions in programs whose scale does not justify building a custom database.

Some design principles

1. Store genotype, phenotype, and pedigree data in an integrated manner
2. Be widely and easily usable
3. Be reasonably memory efficient
4. Be reasonably computationally efficient
5. Be reasonably flexible

What it is (not)

DBDBS is a database interface, not a database. There are many excellent “general purpose” database systems that are applicable to a large number of scenarios. These can be used to store breeding program data, but interacting directly with them can be cumbersome for a non-specialist. Additionally, we can benefit by treating some genetic data in a specific manner, such as recoding genotypes to save memory, that is tedious to perform manually. DBDBS addresses these points by serving as an interface. DBDBS facilitates using the database system for genetic data while the database system does the hard work of actually storing the data. For most, if not all, tasks, it is expected that users will interact directly with DBDBS and not the database system.

DBDBS is written to work with one specific database system, MySQL. MySQL was chosen because it can be used to efficiently store most of the data types of interest to most breeding programs, it is widely and freely available, it is actively supported and updated, there are numerous resources available for learning and troubleshooting, and it is relatively easy to install and (learn to) use.

When you use DBDBS, you use it to log into MySQL (either MySQL on the same computer or on a remote computer), and through DBDBS you are storing and retrieving data from that MySQL system.

Installation

Installing the interface

First, you must install python 3. Development and testing was/is done with version 3.12, but all recent versions are expected to work. You may already have python 3 on your computer, but if not, python 3 can be obtained from https://www.python.org/downloads/. The python package manager "pip" is generally installed with python. If not, you will need to install it. Pip can be used to install DBDBS from the github repository with the command:

```
python -m pip install git+https://github.com/delomast/DBDBS
```

There are a couple of python packages used internally by DBDBS. These should be automatically installed by pip when you install the DBDBS package, but if you want to install them manually, you can. The required packages are "PyQt6" (`pip install PyQt6`) and "mysql-connector-python" version 9.0.0 (`pip install mysql-connector-python==9.0.0`).

Installing MySQL

DBDBS interacts with a MySQL server and stores/retrieves data from the MySQL server. The simplest way to set this up is to run MySQL and DBDBS on the same computer. The freely available community edition of MySQL server can be downloaded from https://dev.mysql.com/downloads/mysql/ and installed using the provided installation wizard. DBDBS has been developed and tested with version 8.5 (the most recent LTS version) and that is the version we would suggest you use. As part of the installation process you will make an account with username "root" and a password. You will need this information (or login information for another account) for DBDBS to interact with MySQL. Other accounts (and the level of permission they have) can be created after installation through MySQL directly on the command line or through a general MySQL administration interface, such as MySQL Workbench. You will want MySQL to either run automatically on startup, or you will need to manually start MySQL whenever you want to use DBDBS.

The program "MySQL Shell" is not required to run DBDBS, but it can make backing up a database a little quicker. Additionally, if you want to interact with MySQL directly for troubleshooting or to perform tasks that DBDBS does not have the functionality for, it can be useful. MySQL Shell can be downloaded from https://dev.mysql.com/downloads/shell/.

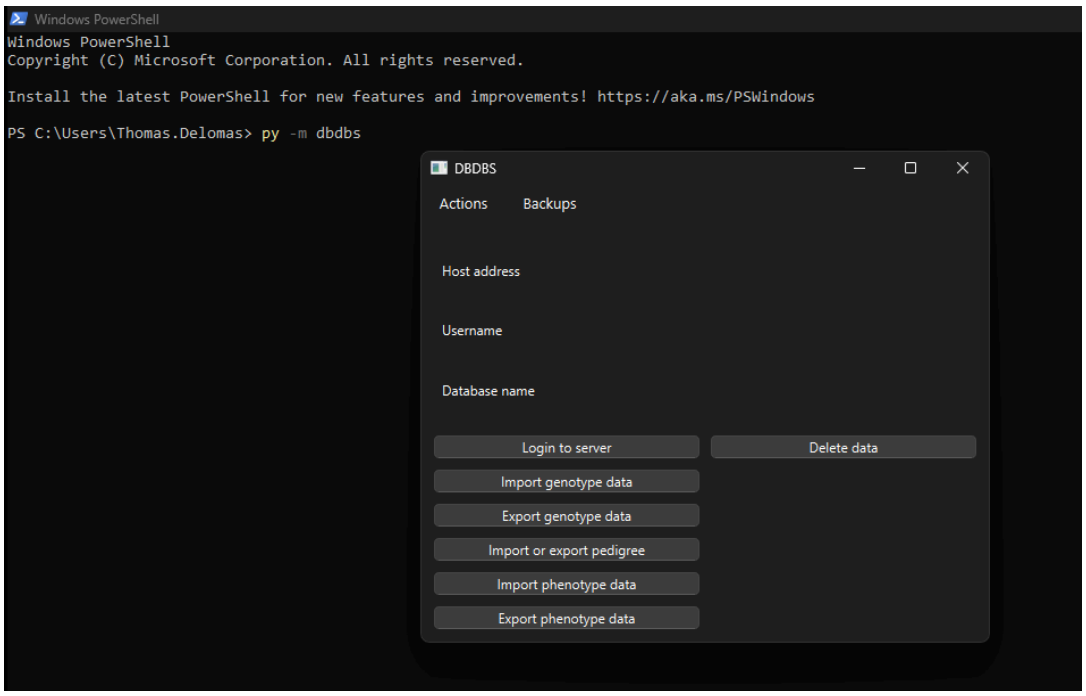
Similarly, a graphical user interface program for general administration of MySQL is not required, but may come in handy. There are many options. One option is MySQL Workbench which can be downloaded from https://dev.mysql.com/downloads/workbench/.

Startup and basics of DBDBS

Starting DBDBS

DBDBS is a graphical interface, but the easiest way to start it is to call it from the command line. This will also ensure that if it runs into an error, the error will printed to the terminal for you to read. After DBDBS is installed, it can be called by executing

```
python -m dbdbs
```



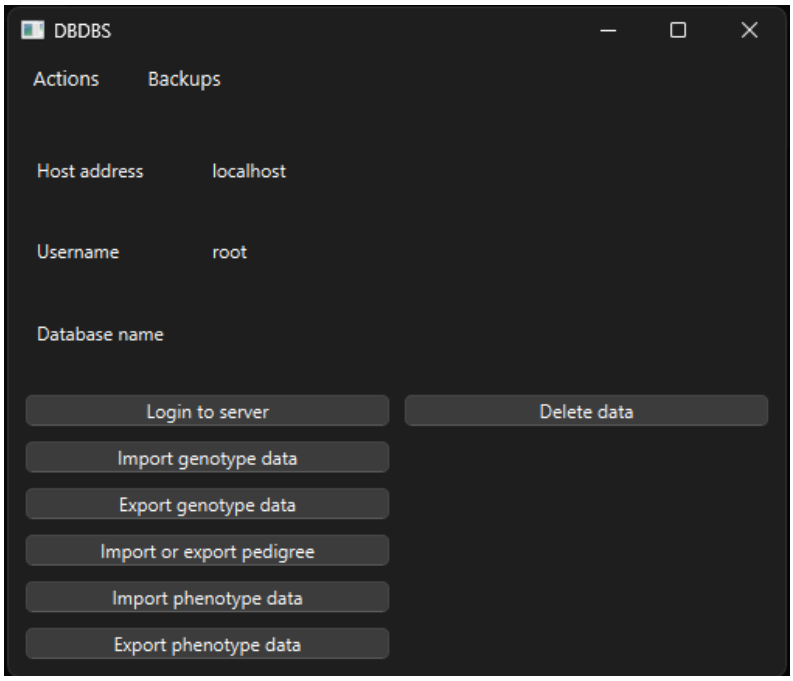
on the command line. This will open the main DBDBS window.

Connecting to a MySQL server

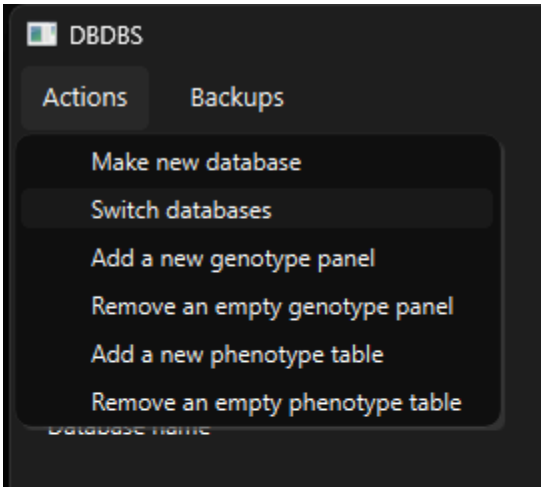
Because DBDBS is just an interface to MySQL, you will first need to connect to a MySQL server. Click the “Login to server” button in the main DBDBS window and fill in the server details. If you are running the server on the same computer DBDBS is running, the address and

username will likely be “localhost” and “root”, respectively. One MySQL server can contain multiple databases. You can leave the “Database name” option blank if you are not logging in to a previously created database. If the connection is successful and the “Save info” checkbox is checked, the server information (except the password) to be stored locally so that a user can select it from the dropboxes in future sessions. Once the server information is entered, click “Connect”.

If the login is successful, the main screen will now show the server address and username.



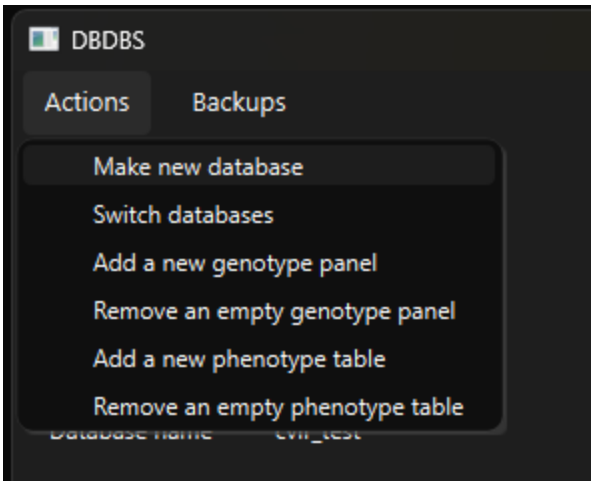
DBDBS can only work in one database at a time. If you already created a database, you could select that database by entering it on the login screen. Otherwise, you can select it by going to the Actions menu at the top of the window and selecting “Switch databases”.



This will open a selection screen with a dropdown menu that shows all the DBDBS compliant databases on the server. This function can also be used to switch between databases on the same server after you have logged in.

Creating a new database

To create a new database, click on the “Actions” menu and select “Make new database”. This will open a window where you can type the name of the database you want to create. Valid names are 1 - 50 characters long, do not start with “intdb” (case insensitive), do not end or begin with whitespace, and do not contain a forward slash, backslash, backtick, single quote, or double quote.



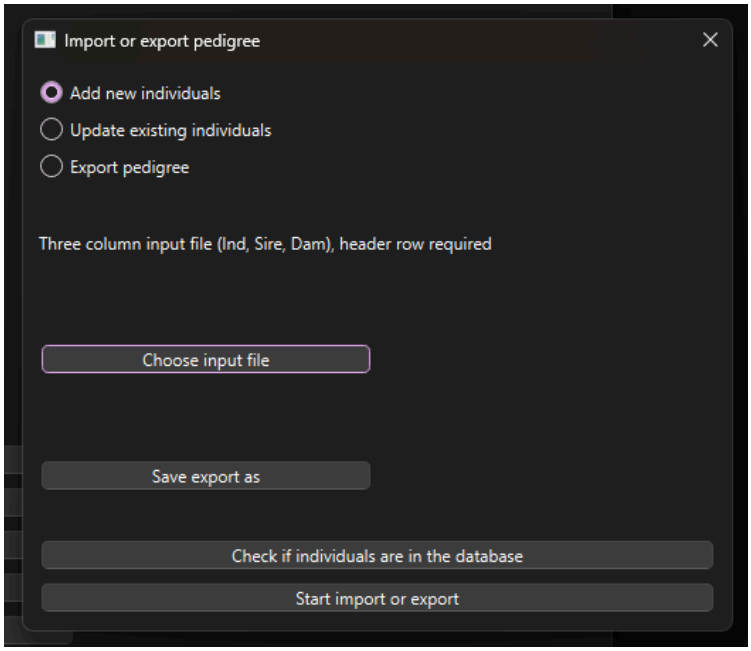
Pedigree data

Pedigree data refers to an individual’s name (identifier) and the names, if known, of its dam and sire. In convention with animal breeding, “dam” refers to the female parent and “sire” refers to the male parent. All individuals with data anywhere in the database must be in the pedigree but the dam and sire is not required to be known.

Import

To import pedigree data, first click the “Import or export pedigree” button in the main window. Then click the radio button for either “Add new individuals” or “Update existing individuals” depending on your task. Only one of these can be performed at a time. Updating existing individuals refers to adding/changing dam and sire information for an individual that is already in the pedigree.

You will need to provide an input file with the pedigree information to import. This should be a tab-delimited text file with three columns. The first (left-most) column contains the individual names, the second specifies the sire, and the third specifies the dam. If the sire or the dam is unknown, that cell should be left blank. Valid individual names are 1 - 255 bytes long (up to 255 characters, but potentially fewer depending on the characters used), and do not contain either a single quote or a backslash. A header row is **required**, but the specific text in the header row is not used. Click on the “Choose input file” button and a



window will open for you to navigate to and select the desired input file.

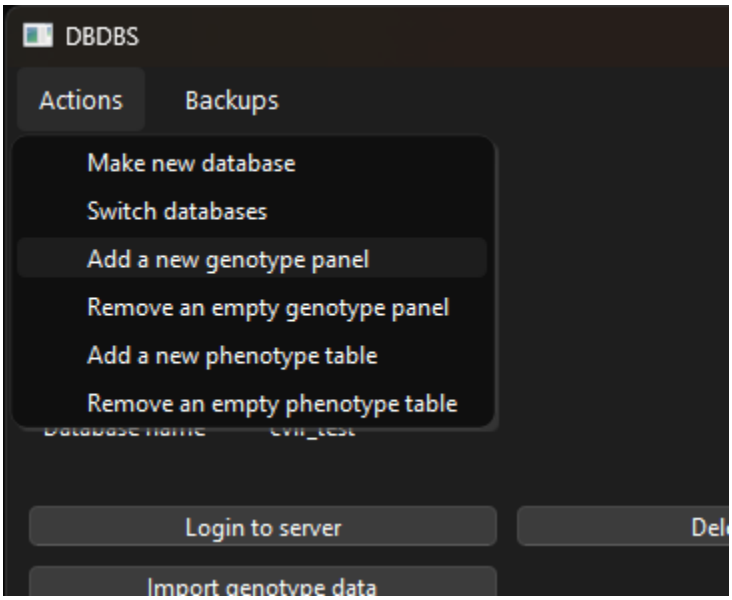
Next, you can click the “Check if individuals are in the database” button. This can help you prevent errors by making sure the individuals are (when updating) or are not (when adding new) in the database already. When you are ready to perform the import, click the “Start import or export” button.

When you are importing sire or dam information (either adding new individuals with this information or updating existing individuals), the sires and dams must already be in the database. Otherwise, you will receive an error message and the data will not be imported. This choice was made (as opposed to having unrecognized parents automatically added) to help prevent errors, such as a mistyped identifier, from entering the database. We realize this is inconvenient when importing data for multiple generations. A quick way to handle importing multiple generations is to first import all individuals (all generations) with the dam and sire columns blank, and then use the “Update existing individuals” option to add the dam and sire information.

Export

To export pedigree data, first click the “Import or export pedigree” button in the main window, and then click the “Export pedigree” radio button. Then, click on “Save export as” and choose a file name and location to save the exported file. If no input file is specified, the entire pedigree will be exported. If you only want to export a subset of the pedigree, specify an input file that has one individual name per line with **no header row**. A pedigree file with the selected individuals *and all their ancestors* (up to 10,000 generations) will be generated. You can click on “Check if individuals are in the database” to make sure the requested individuals are in the database. Then click the “Start import or export” button to export the data. The pedigree will be written as a tab-delimited file with a header and three columns: individual name, sire, dam.

#TODO planned additions: options to export just the selected individuals (no relatives), and selected individuals with all relatives (ancestors and descendants)



Genotype data

Creating a new genotype panel

From the main screen, click on the “Actions” menu and select “Add a new

genotype panel". This will open a screen to specify information about the panel. First, select the type of panel you want to create:

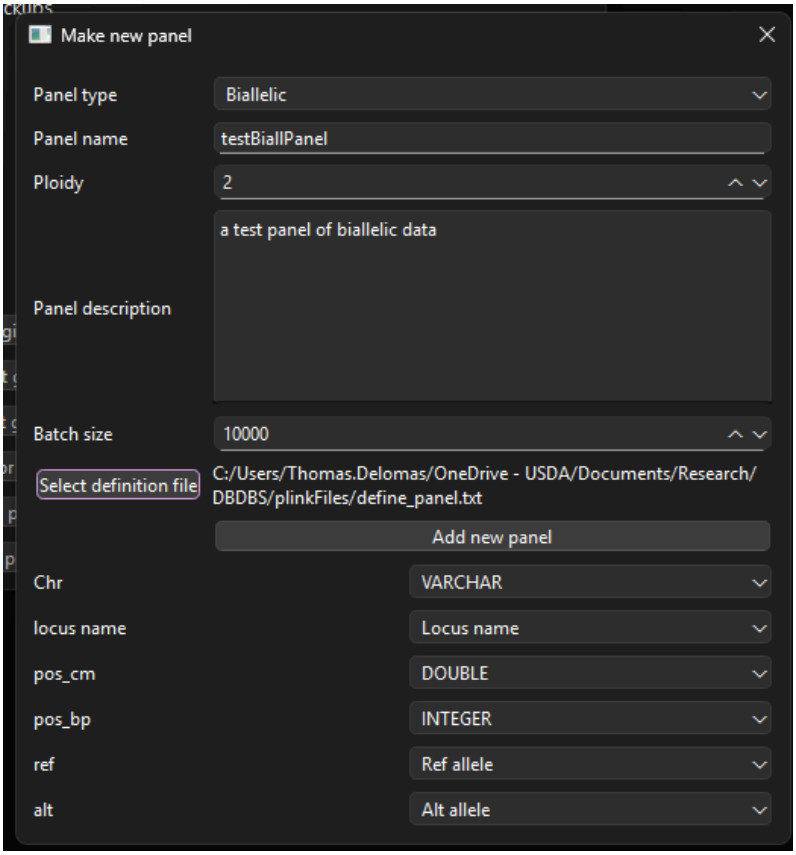
- Biallelic: Each locus has two possible alleles that are specified at the time of panel creation. This is the most storage efficient option and is designed to handle large panels (e.g., SNP array data).
- Multiallelic: Each locus can have multiple possible alleles, and the specific alleles do **not** have to be specified at the time of panel creation. The maximum number of alleles is dependent on ploidy and is such that all genotype states can be represented with one byte. For diploid loci, this equates to 22 alleles. For triploid loci, this is 10 alleles, and for higher ploidies it is less. This option is less storage efficient than Biallelic panels.
- Hyperallelic: Each locus can have multiple possible alleles, and the specific alleles do **not** have to be specified at the time of panel creation. The maximum number of alleles is such that all allele states can be uniquely represented with one byte. This equates to a maximum of 255 alleles per locus regardless of ploidy. This option is less storage efficient than Multiallelic panels.

Choose a short panel name. Valid names are 1 - 50 characters long, do not start with "intdb" (case insensitive), do not end or begin with whitespace, and do not contain a forward slash, backslash, backtick, single quote, or double quote.

Specify the ploidy. All loci in a panel are given the same ploidy.

Type a panel description. This is stored internally for a user to later reference, but it is not otherwise used. It can be very long (up to 64 KB).

Choose a batch size. This controls the number of loci that are created in the panel at a time. This turned out to be a pretty useless setting. If you are very memory limited and are uploading a large amount of descriptive data for each locus (unlikely to be the case), consider using a value in the low thousands. Otherwise, make the value equal or larger than the number of loci in your panel (or just leave it at the default).



Select the definition file. The definition file must be a tab-delimited file with a **required** header. A user can optionally upload a number of extra columns with information about the loci. These extra columns are not used, only stored for a user to later view. After selecting the definition file, the column names will be displayed along with dropboxes for you to indicate how each column should be treated:

- Locus name: this is the name of the locus and is a required column type. Locus names *must* be unique within a panel but do not have to be unique across panels. For example, if you genotype diploids and triploids with array X, you could create two panels in the database, array_X_2n and array_X_3n, with identical locus names. Locus names must be 1 - 2000 bytes in length and cannot contain a backslash (\) or a single quote (‘).
- Ref allele: This is the text representation of the reference allele. It is required for biallelic panels and not allowed for others. Alleles must be less than 255 bytes, at least one character in length, and not contain a single quote, comma, backslash, or whitespace. It is recommended that all ref alleles be one character in length.
- Alt allele: This is the text representation of the alternate allele. It has the same restrictions as the “Ref allele” column.
- Alleles: This is a comma separated list of known alleles. This is an optional column for multiallelic and hyperallelic panels and is not allowed for biallelic panels. Alleles must be less than 255 bytes, at least one character in length, and not contain a single quote, comma, backslash, or whitespace. *The alleles column is not required because you have the option to add alleles to these panels when new alleles are detected during a genotype import.*
- VARCHAR: This column should be treated as short text data about the locus.
- INTEGER: This column should be treated as integer data about the locus.
- DOUBLE: This column should be treated as numeric data about the locus.
- DATE: This column should be treated as a date with the *required* format YYYY-MM-DD.
- TEXT: This column should be treated as longer text data about the locus.

There are different required columns depending on the type of panel you are creating:

- Biallelic: Locus name, Ref allele, Alt allele required
- Multiallelic: Locus name required
- Hyperallelic: Locus name required

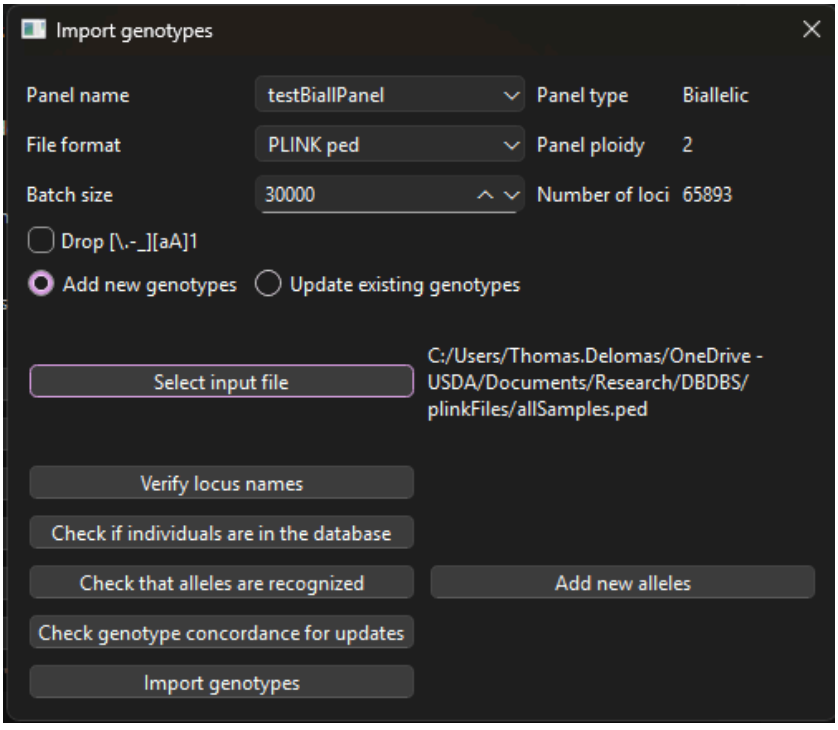
After specifying all the columns, click “Add new panel” to add the panel to the database.

Importing genotype data

To store genotype data for a given individual, that individual must be in the pedigree. The genotype import function will **automatically add individuals to the pedigree** (with missing dam and sire information) as needed when importing genotypes. For this reason, it is recommended to always run the “Check if individuals are in the database” check (see below) prior to import to ensure the results match your expectations.

From the main screen, click the “Import genotype data” button. Then, select the panel from the dropbox that you want to import data to. The panel type, ploidy, and number of loci will be shown on the right side of the window. Next, choose the file format of the file you are uploading:

- 2col: This is a tab-delimited file with a **required** header row. The first column contains the individual names. The following columns are one column per allele with loci represented by adjacent columns. For diploid data, this is a “2 columns per call” format. For other ploidies, the number of columns per call will vary with ploidy. After the first column, the header row should contain the locus names with an optional suffix of the format (.-_)(aA)1 for the first allele column in a locus, (.-_)(aA)2 for the second, and so on. For example, a header row could contain “IndName locus1.A1 locus1.A2 locus2.A1 locus2.A2” or simply “IndName locus1 locus1 locus2 locus2”. Internally, only the header of the first column for each locus is used to identify the locus. The following column(s), depending on the ploidy, are assumed to correspond to the preceding locus. If the optional suffix is included, make sure to check the “Drop [\.-_][aA]1” checkbox. **Missing genotypes are specified by empty cells.**
- PLINK ped: This is a “ped” text file as specified by the PLINK manual and there must be a corresponding “map” file with the same name but ending with “.map” instead of “.ped”.
- long: This is a one row per call format. It is a tab-delimited text file with a **required** header row, but the specific text in the header row is not used (the header must have the correct number of columns). The first column contains individual names, the second column contains locus names, and the following columns contain the called alleles. The number of allele columns should be equal to the ploidy. For efficiency, it is recommended that genotypes for a given individual be in adjacent rows (e.g., the file should ideally be sorted by individual). **Missing genotypes are specified by empty cells.**
- #TODO add PLINK binary



If using the “long” format, select the desired “batch size”. This is the maximum number of rows to read at once. The “batch size” argument is not used for other formats. Then select whether you are

adding new genotypes (the individuals do not already have genotypes for this panel in the database) or updating (overwrite existing) genotypes. Only one (add new or update) can be performed at a time.

Click the “Select input file” button to select an input file. The input file does not need to contain all loci in the panel. When adding new genotypes, missing loci are recorded as missing genotypes for all individuals in the uploaded file. When updating genotypes, missing loci are left unchanged.

There are several optional, but recommended, checks to execute prior to attempting to import data.

- Verify locus names: Check whether locus names in the input file match those expected for the selected panel
- Check if individuals are in the database: Check whether individuals are 1) in the pedigree and 2) already have genotypes in the database for the selected panel
    - Remember that importing genotype data will automatically add an individual to the pedigree. In order to prevent misspelled individual names from creating unneeded entries in the pedigree, run this check before importing data to make sure the status of individuals being in the pedigree matches your expectation.
- Check that alleles are recognized: Check that the alleles in the input file match those expected.
    - For multiallelic and hyperallelic panels, if this check finds new alleles then you may click the “Add new alleles” button to add them to the list of valid alleles for the respective panel/loci.
- Check genotype concordance for updates: This will summarize the concordance between the genotypes already in the database and the genotypes being uploaded. It is a useful check prior to updating (overwriting) genotypes.

After running the desired checks, click on “Import genotypes” to complete the import.

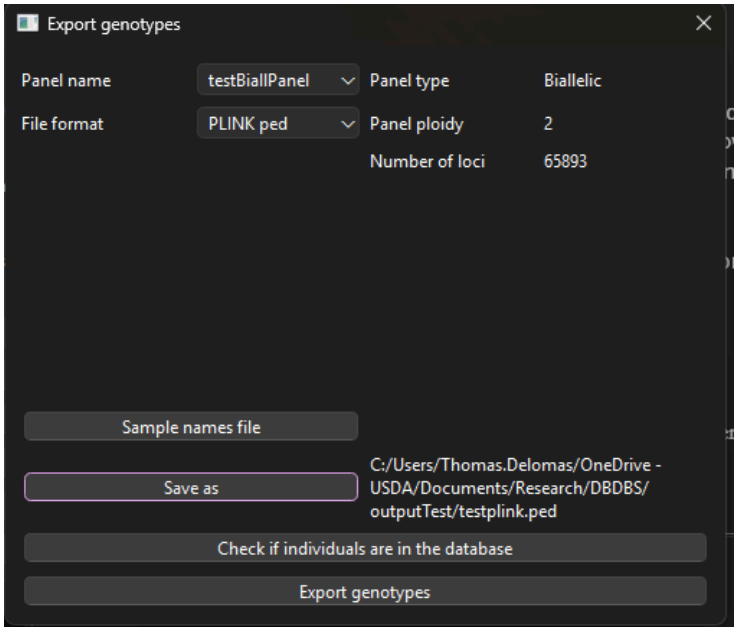
If a problem that cannot be recovered from is encountered, the import will be canceled and the interface will close. An error message will likely be displayed on the command line where you started DBDBS.

Exporting genotype data

From the main screen, click the “Export genotype data” button. Then, select the panel from the dropbox that you want to export data from. The panel type, ploidy, and number of loci will be shown on the right side of the window. Next, choose the file format of the file you want to create. The supported formats are described in the Importing genotype data section of this manual. For the PLINK format, dummy values are used for the metadata fields.

Optionally, click the “Sample names file” button to select a file containing the names of the samples you want to export genotypes for. If no file is selected, all genotypes for the selected

panel are exported. The input file is a tab-delimited file with a **required** header row and the first column should contain the individual names. Only one column is required, and any other columns are ignored. The intent is to allow phenotype data to be exported (see [add link]) and then for that phenotype export file to be used as an input here to export genotypes for those individuals.



Click “Save as” to specify the output file name.

The “Check if individuals are in the database” button can be used to verify that the requested individuals (in the optional sample names file) have genotypes in the selected panel.

Click “Export genotypes” to start the export.

Phenotype data

Phenotype tables can be used to store information on individuals or full-sib families (unique combination of sire and dam). As one may presume, the functionality is designed with storing phenotypes in mind, but they are flexible so that a user can choose to store other data (for example, year class, farm location) in them as well. There are two broad types of phenotype tables: individual level and family level.

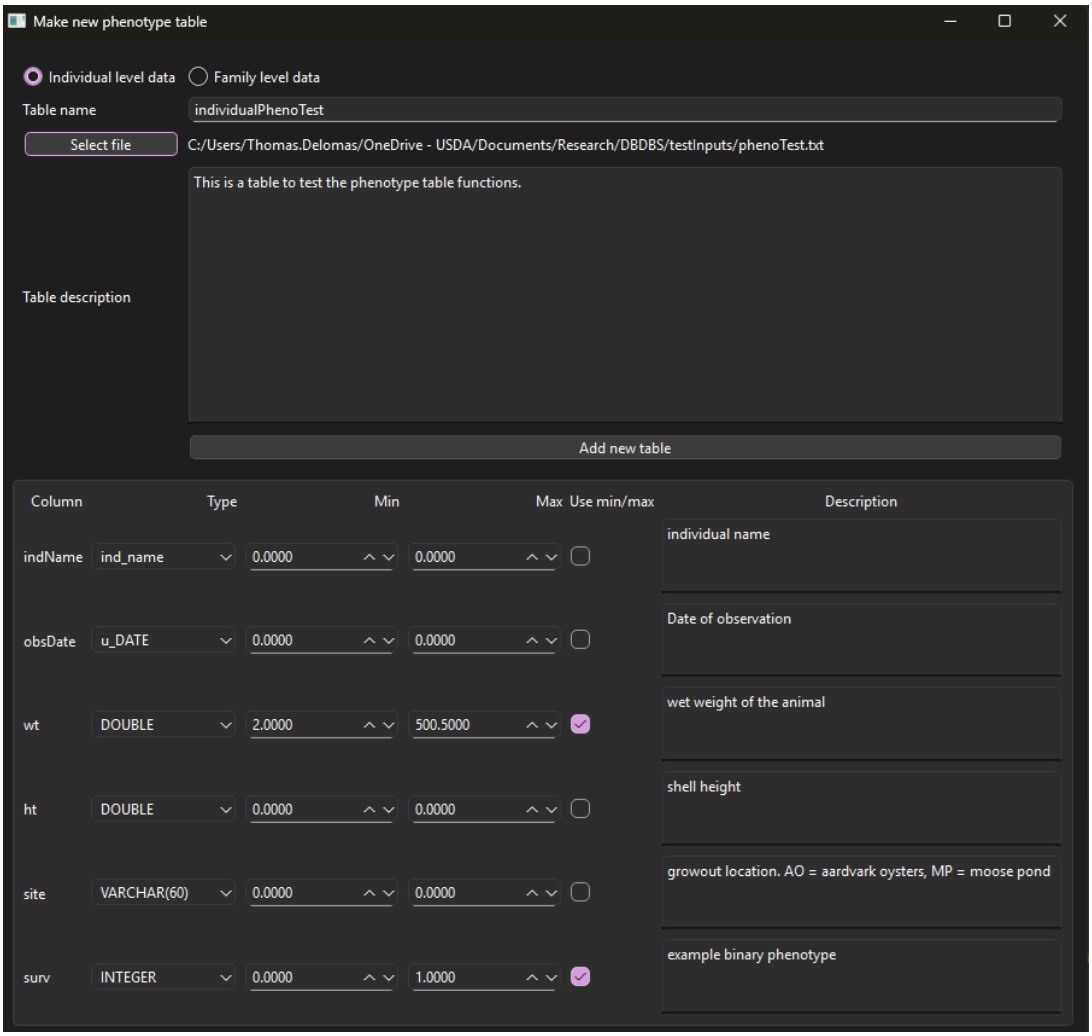
Individual level tables store data on individuals. Each row of these tables has a unique combination of individual name and date or datetime. Family level tables store data on full-sib families. Each row of these tables has a unique combination of sire, dam, and date or datetime. The inclusion of a date/datetime variable allows repeated measurements to be recorded for a given individual or family. Among other uses, the family level tables allow efficient storage of data that is binary at the individual level (e.g., survival over a certain time period).

Users can define phenotype tables to best suit their program structure. For example, to store weights observed on individuals raised at different farms, a user may choose to make one phenotype table that includes a “farm” variable. A different user may choose to make a separate phenotype table for each farm.

Creating a new phenotype table

To create a phenotype table, first go to the “Actions” menu and click on “Add a new phenotype table”. Then select whether you are creating an individual level or family level table and add a written description of the table as you see fit (it can be very long, up to 64 KB).

Click on “Select file” to select a definition file. The definition file is typically just a file that you would use to import phenotype data to the database. It is a tab-delimited file and only the header row is used to define the panel. The header row should contain the names of the variable you want to include in the phenotype table. After selection, the window will maximize and populate with a list of the variable names it detected. Note that you can resize the top and bottom sections independently by hovering your mouse below the “Add new table” button until the “resize” cursor is shown and then clicking and dragging.



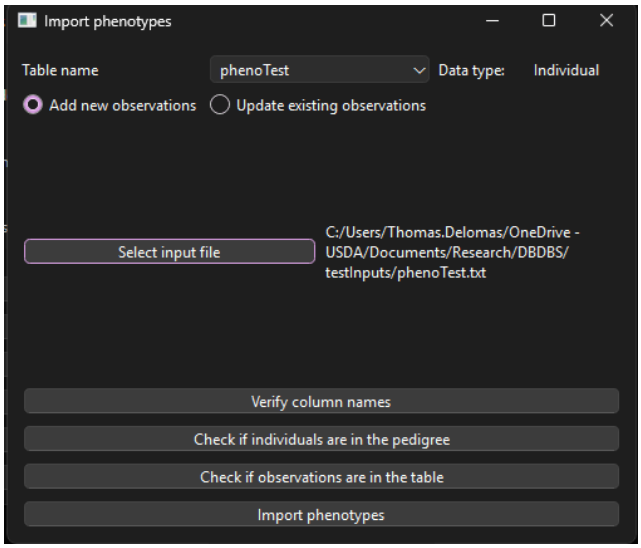
Use the dropdown boxes to assign a “type” to each variable. The types are:

- ind_name: the name of the individual the row corresponds to. Required for individual level tables and not allowed for family level tables.
- sire_name: the name of the sire of the family the row corresponds to. Required for family level tables and not allowed for individual level tables.
- dam_name: the name of the dam of the family the row corresponds to. Required for family level tables and not allowed for individual level tables.
- u_DATE: the date that will be used to enforce unique date and individual name (or sire x dam) combinations for rows. Intended to be the date the observation was made. One and only one column must be either u_DATE or u_DATETIME.
- u_DATETIME: the date and time that will be used to enforce unique datetime and individual name (or sire x dam) combinations for rows. Intended to be the datetime the observation was made. One and only one column must be either u_DATE or u_DATETIME.
- INTEGER: integer data
- DOUBLE: numeric data
- VARCHAR(60): short text data with entries up to 60 characters long
- VARCHAR(1000): long text data with entries up to 1000 characters long
- TEXT: very long text data with entries up to 64 KB long
- DATE: data that is in the form of a date, but not to be used to enforce the unique row requirement
- DATETIME: data that is in the form of a date, but not to be used to enforce the unique row requirement

For the columns that are defined as INTEGER or DOUBLE, minimum and maximum values can optionally be chosen by inputting them in the “Min” and “Max” fields and checking the “Use min/max” box. DBDBS will reject values that are outside this range when phenotype data is attempted to be uploaded through DBDBS.

A text description can be added for each column. This is not used by DBDBS, only stored for a user to later refer to. Each description can be very long (up to 64 KB).

After adding all the needed data, click on “Add new table” for the phenotype table to be created.



Importing phenotype data

To store phenotype data for a given individual, that individual must be in the pedigree. The phenotype import function will **automatically add individuals to the pedigree** (with missing dam and sire information) as needed when importing phenotypes. For this reason, it is

recommended to always run the “Check if individuals are in the pedigree” check (see below) prior to import to ensure the results match your expectations.

From the main screen, click on the “Import phenotype data” button. A new screen will open and you can select the table you want to import data into using the dropdown box. Whether the table is an individual or family level table will be shown on the right side of the screen.

Choose whether you are adding new observations or updating existing observations and then click on “Select input file” to select the file containing the data you want to import. The input file is a tab-delimited file with a **required** header whose column names match the column names used when creating the phenotype table. You can verify that the column names match what is expected by clicking the “Verify column names” button. Similarly, you can use the other buttons to check if the individuals are already in the pedigree and if the observations (defined as a unique individual or dam x sire AND date/datetime combination) are already in the table.

The format for date and datetime columns is very specific. Dates must be expressed as “YYYY-MM-DD” and datetimes must be expressed as “YYYY-MM-DD hh:mm:ss” (the whitespace character between the day and the hour is a space NOT a tab) with hours ranging from 00 to 23 (24-hour clock). For example, an input file with a date may look like:

indName	obsDate	wt
Ind1	2025-05-22	34.1
Ind2	2025-05-22	22.4

and an input file with a datetime may look like:

indName	obsDateTime	wt
Ind1	2025-05-22 15:05:33	34.1
Ind2	2025-05-22 15:06:20	22.4

Empty cells are treated as missing values.

To complete the import, click on the “Import phenotypes” button.

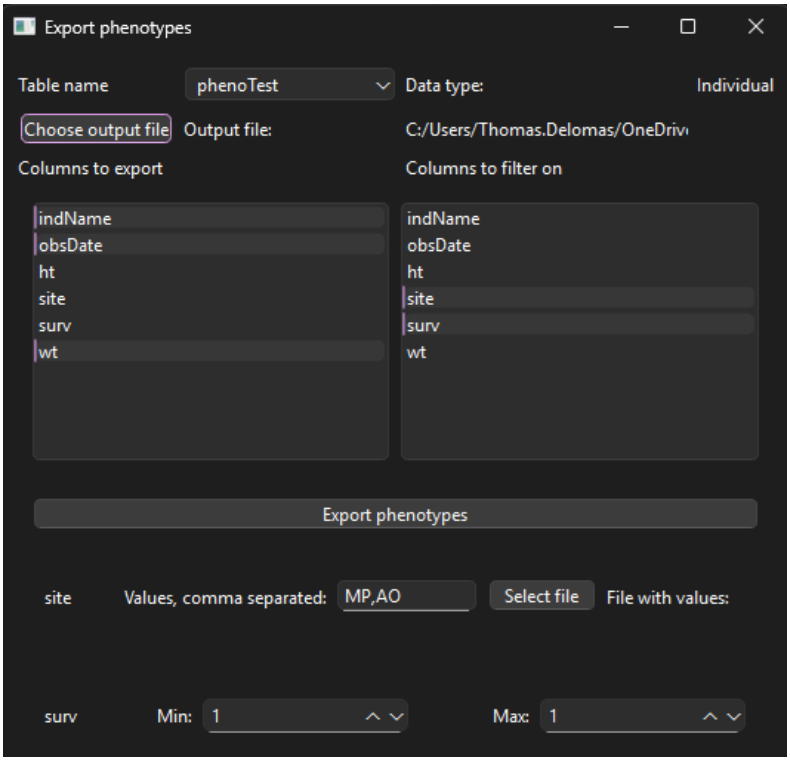
Exporting phenotype data

To export phenotype data, click on the “Export phenotype data” button on the main screen. Then, select the table you want to export data from using the dropdown box and specify an output file name by clicking the “Choose output file” button.

There will be two selection boxes containing all the column names in the phenotype table. In the selection box on the left, highlight the columns you want in the export. Note that you can reorder them by clicking and dragging to change the order in the output file (top to bottom in the box becomes left to right in the file). Multiple adjacent columns can be selected by clicking one, holding the shift key and clicking a second to select all columns in between the two. Multiple

columns can also be selected (regardless of whether they are adjacent) by holding the ctrl key while clicking.

In the box on the right, select the columns, if any, that you want to use to filter the exported data. The screen will then display the selected columns and input options for establishing conditional filters. For string (text data) columns, you can specify the values to include in the export by either typing a comma separated list, or by specifying the location of a file with one value per line (no header line). Whether these filters are case sensitive depends on the MySQL settings of your server. For numeric columns, you can specify a minimum and maximum value to include in the export. For date and datetime columns, you can specify a date/time range to include in the export.



Missing values are exported as empty cells. When filtering on a column, rows with missing values for that column will be excluded from the export.

To complete the export, click the “Export phenotypes” button.

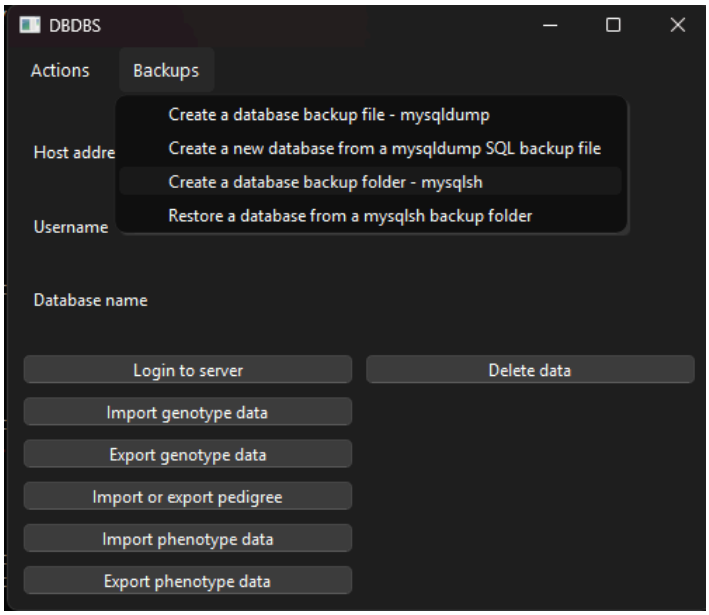
Backing up the database

Backing up a database periodically or before potentially consequential actions is a good practice. Since DBDBS is just an interface to MySQL, a user may choose to employ any of the multiple available options for backing up MySQL databases. Additionally, there are two ways that DBDBS can be used to create a backup of a database.

Using MySQL Shell through DBDBS

The program “MySQL Shell” can create and import database backups. If it is installed on your computer, you can use it directly or you can use it through DBDBS. If the MySQL Shell executable (`mysqlsh`) is not found on your system’s path, you will be prompted to locate it.

Creating the backup



From the main screen, click on the “Backups” menu and select “Create a database backup folder - mysqlsh”. A warning will be displayed and then a location specification box will appear for you to choose the location and name of a **new folder** that will be created and contain the backup files. The backup will be created and then a message notifying you that the backup is complete will be displayed.

Importing the backup

To import a backup created by MySQL Shell through DBDBS, there must not be a database already existing with the same name as the database that the backup represents. If there is, we recommend you first copy the existing database of that name to a different name and then delete the original copy. To copy the existing database to a different name, a user can use the “Migration Wizard” in the program “MySQL Workbench” to migrate the database onto the same server but with a different name. The original copy of the database can then be deleted either through MySQL Workbench or through the MySQL command line interface directly by using the `DROP DATABASE` command.

To execute the import, click on the “Backups” menu and select “Restore a database from a mysqlsh backup folder”. A warning will be displayed and then a navigation window will open so you can select the folder containing the backup files. After the import has finished, a message will be displayed. You can then use the “Switch database” function to switch to the imported database.

Note that during the import, MySQL Shell creates a “progress file” in case the import is interrupted. If the import is restarted after benign interrupted, the progress file will automatically cause the import to restart where it left off. If you want the import to restart at the beginning, simply delete the “load-progress…” file that was created in the backup folder.

Using mysqldump through DBDBS

The program “mysqldump” is less efficient than MySQL Shell, but it is typically installed along with MySQL and there are some use cases where it may be preferred. If it is installed on your computer, you can use it directly or you can use it through DBDBS. If the executable (`mysqldump`) is not found on your system’s path, you will be prompted to locate it. It is typically in the “bin” folder of the MySQL Server installation.

Creating the backup

From the main screen, click on the “Backups” menu and select “Create a database backup file-mysqldump”. A warning will be displayed and then a file specification window will appear for you to choose the location and name of a **new file** that will be created and contain the backup information. The backup will be created and then a message notifying you that the backup is complete will be displayed.

Importing the backup

From the main screen, click on the “Backups” menu and select “Create a new database from a mysqldump SQL backup file”. A warning will be displayed and then you will be asked for the name of the new database to create. A file specification window will then appear for you to choose a backup file created by mysqldump. The new database will be created and the backup will then be loaded into it. A message notifying you that the import is complete will be displayed.

The import process accesses the MySQL directly on the command line. If the MySQL executable `mysql` is not found on your system’s path, you will be prompted to locate it. It is typically in the “bin” folder of the MySQL Server installation.

The import process is simply the execution of all commands in the input SQL file. There are no checks to determine if the file was created by mysqldump. If the SQL file was created by another means, such as a malicious actor, the effects on your database could be detrimental. Only use files from sources you trust.

Delete data

Sometimes we need to delete data that was uploaded in error. This is not often the case, and there is no “undo” functionality in DBDBS. To prevent accidental deletion of data, the DBDBS functions for deleting data are purposefully a little inconvenient.

To delete data, click the “Delete data” button on the main window. A warning is displayed and then a new window is opened. Select the type of data you want to delete. Note that before deleting an individual from the pedigree, all genotype and phenotype data for that individual must be deleted and that individual must not be listed as a dam or sire in the pedigree or of any family in a family phenotype table.

If you are deleting phenotype or genotype data, select the panel or table you want to delete data from.

Select an input file specifying the individuals (deleting genotype or pedigree data) or observations (deleting phenotype data) you want to delete. The input file is a tab-delimited file with **no header row**. The format of the input file depends on the type of data you are deleting.

- Pedigree and genotype data: one individual name per line
- Individual level phenotype data: first column is individual name second column is date(time)
- Family level phenotype data: first column is sire name, second is dam name, third is date(time)

For deleting phenotype data, remember that in the input file dates and datetimes must have the format “YYYY-MM-DD” and “YYYY-MM-DD hh:mm:ss” (the whitespace character between the day and the hour is a space NOT a tab), respectively.

To check that your input file is formatted correctly and the expected amount of data will be deleted, click on the “Check input file” button. It will tell you how many rows will be deleted.

To permanently delete the data, click the “Delete data” button.

Helpful tips

- To export phenotype, genotype, and pedigree data for a set of individuals, first export the phenotype data with the individual name as the first (left-most) column, then use that as an input to export genotype and pedigree data for those individuals.
- **There is no “undo” functionality available in DBDBS.** All actions should be treated as permanent. Backing up the database at regular intervals and before performing consequential actions (e.g., deleting data) is recommended. Advanced users of MySQL may be able to “undo” actions by directly using MySQL if the MySQL server is configured in a particular way (generating an appropriate binary log) **prior** to performing the action that a user wants to undo. This is outside the current scope of DBDBS. Users that require this option are encouraged to learn more about the MySQL database system.
- Check https://github.com/delomast/DBDBS periodically to ensure you are running the most recent version of DBDBS
- The preferred method to report bugs, ask usage questions, or request new features is at https://github.com/delomast/DBDBS/issues
    - Bug reports, questions, and feature requests can alternatively be sent via email to thomas.delomas@usda.gov
- Many of the maximum lengths are expressed in bytes. This is because that is the way MySQL enforces some length limits. Generally, latin letters are 1 byte in length but other characters may require more bytes.
- DBDBS is probably an abbreviation for Directed Breeding Database System
- When exporting phenotype data and filtering on string (text data) columns, the case sensitivity depends on the preferences set in your instance of MySQL. Typically, the default is for case-insensitive filtering. To change this, you can work in MySQL directly to change the “collation” to a case sensitive collation. For example, default is often

“utf8mb4_0900_ai_ci”, which is case insensitive, but you could change it to “utf8mb4_0900_as_cs”, which is case sensitive.